BVS1 · Winter Semester 2026/2027 · TH Köln · ICCT

Exercise 4 — One Trap Path

Except where otherwise noted, © 2026 Stavros Kousidis · CC BY-SA 4.0

Planned time: 90 minutes

Guiding Question

> How does control enter the OS through the hardware, and what must be saved so that the same execution continues afterwards?

What You Should Be Able to Do Afterwards

- port the Exercise 2 application to the trap path and explain each change;
- draw the lifetime of a run with the CPU in the middle, distinguishing traps from returns;
- trace one `ecall` from `permute()` into the OS and back, naming the active stack and the location of the saved state at each step;
- explain why a trap frame saves all registers and the `pc`, while the CP1 context saved only `ra`, `sp` and `s0`–`s11`;
- diagnose failures of the trap path from their observable behaviour.

Starting Point — 5 Minutes

You need your **Exercise 2** files `process.h` and `main.c` on your local `exercise-02` branch. Commit any open work there first.

On the physical host, enter the VM:

```
multipass shell bvs1
```

Inside the VM, in your existing `teaching-bvs1-code` clone, get the exercise branch and bring your two files:

```
cd ~/teaching-bvs1-code
git status
git fetch origin
git switch exercise/04-trap-path
git switch -c exercise-04
git checkout exercise-02 -- process.h main.c
```

`git status` must show no uncommitted changes before you switch. Work on your local `exercise-04` branch. Do not push student work to the course repository. The course branch contains the Exercise 4 scaffold only; the last command copies your two Exercise 2 files into your local branch.

Bring only these two files. Do not bring a `process.c` from your CP1 work: the OS comes from `lib/libkernel.a` in this exercise, and a second implementation would clash with it.

If your Exercise 2 files are not ready, work with a partner's copy. The reasoning tasks do not depend on your own implementation.

1. Understand the Trap Path — 10 Minutes

The branch provides:

```
trap.h, trap.S       struct trap_frame, trap path, return path
riscv.h              CSR access, mcause codes, CLINT timer helpers
syscall.h, syscall.c system-call numbers, process_yield(),
process_exit()
uart.h, uart.c       UART output, now shared by the OS and the process
lib/libkernel.a      the OS: process table, trap_handler(), os_run()
```

`context.h` and `context.S` are gone.

Work in pairs and open `trap.S`. One of you explains the **trap path** line by line, from `trap_entry` to `call trap_handler`; the other explains the **return path**, from `os_start` to `mret`. For each line, say what it does and why it must be there, at this position. The listener asks "why here?" until both of you agree. Lines that repeat (`sd x1` … `sd x30`) count as one.

Then answer together from the files:

1. In `trap.h`, at which byte offset is register `x5` (`t0`) stored? At which offset is the `pc`?
2. In `trap.S`, which register is swapped with `mscratch` first, and why must trap entry do this before it can store anything?
3. In `trap.S`, which instruction selects the OS stack? What does `sp` select before that instruction?
4. In `syscall.c`, which register carries the system-call number, and which numbers do `yield` and `exit` use?
5. Which part of `trap.S` implements the **trap path** into the OS, and which part implements the **return path** to a process? Which instruction ends the return path?

Evidence: five short answers, each pointing to a line in the file.

2. Update Your Process Interface — 10 Minutes

`lib/libkernel.a` is compiled against this representation. Change your Exercise 2 `process.h` in exactly three places:

1. include `"trap.h"` instead of `"context.h"`;
2. replace the field `struct context context;` by `struct trap_frame frame;` at the same position;
3. remove the declaration of `process_run()` and declare `void os_run(void);` instead.

Keep `STACK_SIZE`, `process_entry_t`, the states `READY`, `RUNNING`, `FINISHED`, and the field order: entry, stack, frame, state, exit status.

Evidence: `git diff -- process.h` and one sentence on why the field order still matters.

3. Port `main.c` and Run — 15 Minutes

Keep `worker()` and `permute()` unchanged. Change three things:

1. delete your own `uart_putc()` and `uart_puts()` and add `#include "uart.h"` (the OS now prints too, so UART output lives in the shared `uart.c`);
2. declare the process object as `static struct process p;`;
3. replace everything after `process_create(&p, worker);` by a single call to `os_run();`.

Before you run it, predict the complete output. Which lines come from the process, and which from the OS? Then:

```
make
make run
```

Exit QEMU with **Ctrl-a**, release, then **x**.

Evidence: your prediction, the observed output, and `git diff -- main.c`. Mark each output line as *process* or *trap handler*.

4. Draw Your Lifetime — 10 Minutes

Draw the lifetime of the run you just observed, in the style of Lecture 5: three columns **OS**, **CPU** and **process**. Draw the CPU as a box with the two mechanisms **trap** and **return**; draw a vertical line for the OS and the process only while that execution runs.

Your diagram must show:

- the first start of the process;
- the `yield` as a trap into the OS and a return to the process;
- the `exit` as a trap into the OS;
- which `os:` output line is printed during which OS bar.

Then mark with a dashed line where the OS line of the Lecture 3 lifetime continued while the process ran. Explain in one sentence why that line is missing now, and in one sentence why the first start counts as a return.

Evidence: your diagram and the two sentences.

5. Trace One `ecall` — 20 Minutes

Disassemble the system-call stub and the trap entry:

```
riscv64-unknown-elf-objdump -d --disassemble=process_yield
build/hello-bvs1.elf
riscv64-unknown-elf-objdump -d --disassemble=trap_entry
build/hello-bvs1.elf
riscv64-unknown-elf-nm -n build/hello-bvs1.elf | grep -E ' [bB] '
```

Note the address **E** of the `ecall` instruction in `process_yield`.

Complete the table for the yield inside `permute()`:

Step	Instruction or event	`pc` points into	`sp` selects	The process's state is in
1	`permute()` calls `process_yield()`		process stack	CPU registers
2	`ecall` at E executes			
3	first instruction at `mtvec`	`trap_entry`		
4	after the stores, before `la sp`			
5	inside `trap_handler()`			
6	`mret` has executed			
7	`process_yield()` returns	`permute()`		

Then answer:

1. Which value does `mepc` hold at step 3, and therefore `p.frame.pc` right after trap entry? Which continuation address does `p.frame.pc` hold when the frame is restored at step 6? Use E, and explain why the two differ.
2. Why does the OS read the system-call number from `p.frame.regs[17]` and not from the live `a7` register?
3. CP1 saved 14 values. Name one register the trap frame saves that CP1 did not save, and describe a situation in which losing it would break the interrupted code.
4. The `nm` line filter lists BSS-related symbols. One of them is your static `p`; depending on the compiler it is named like `p.0` or `main.p`. Compare its address with `_stack_top`. Is `p` stored inside the OS stack?

Evidence: the completed table with E filled in, and four short answers.

6. Diagnose — 15 Minutes

For each case, state the wrong assumption, predict the observable behaviour, and explain the correction.

1. **Try it:** change `static struct process p;` back to `struct process p;`, rebuild, and run. The output may still look correct. Find out whether it is:
   - with `riscv64-unknown-elf-objdump -d --disassemble=main build/hello-bvs1.elf`, find how much stack `main()` reserves and which address it passes to `process_create()` in `a0`: that is where `p` now lies;
   - with `--disassemble=trap_handler`, find how much stack the handler reserves when every trap starts it at `_stack_top`;

- name the fields of `p` that lie in the handler's range, and explain why the run can look correct and is still wrong.

Then restore `static`.

2. The OS's `trap_handler()` forgets `trapped->pc += 4` for system calls.
3. **Try it:** insert `__asm__ volatile (".word 0");` as the first statement of `worker()`, rebuild, and run. Locate the reported address with `objdump -d --disassemble=worker`. Then remove the line.

For cases 1 and 3, report what you observed and the evidence you found. For case 2, a prediction with justification is enough.

Evidence: three diagnoses with predicted or observed behaviour, not just true/false answers.

Exit Question — 5 Minutes

In four to six sentences, explain:

> What happens between `permute()` calling `process_yield()` and `process_yield()` returning into the same `permute()` call?

Use **`ecall`**, **trap**, **`mepc`**, **trap frame**, **OS stack**, **return**, and **`mret`**. Then explain in one sentence why the first start of a process is also a return, and complete the sentence: *`_start` is the CPU's entry, `main()` is …, and the trap vector is …*